LinkedIn Assistant: An Explainable, Open-Weight LLM Framework for Job Matching

Aryan Makhija amakhij9@asu.edu
Krish Dhaval Naik knaik13@asu.edu
Ashmit Amit Pai apai10@asu.edu
Himanshi Sunilbhai Ruparelia hrupare1@asu.edu
Aditya Umesh Upadhyay aupadh47@asu.edu
Arizona State University, Tempe, AZ, USA

***Abstract*—Career assistants usually wire resume parsing, job matching, and knowledge-graph (KG) reasoning to one proprietary LLM, leaving two questions open: would a cheaper open-weight model work as well, and can the system justify *why* a job fits? Our assistant keeps the agent harness fixed while a model router sends each sub-task to an open-weight model served locally (Ollama) or through hosted inference APIs (Groq, OpenRouter), with a proprietary model kept only as a measured baseline. A minimal job-market ontology guides extraction, so every skill, requirement, and company fact becomes a typed Neo4j node linked to its source text. Matching is a cascade from classical BM25, TF-IDF, and Jaccard baselines through dense retrieval with Reciprocal Rank Fusion to a LambdaMART ranker with graph features, and explanations may only state what the graph can cite. We report extraction F1, nDCG@10, explanation faithfulness, a user study, and, as the headline result, the accuracy-cost-latency trade-off across model tiers.**

***Index Terms*—open-weight LLMs, knowledge graphs, ontology-guided extraction, hybrid retrieval, learning to rank, LoRA, explainable job recommendation**

I. Problem Definition

Keyword filters reject a CV that says "PyTorch" when a posting asks for "deep learning frameworks"; embedding-only recommenders recover such matches but cannot explain them, and neither tells the candidate what to change or whom to contact. Reviews of job recommender systems report the same gaps in explainability and in support beyond matching [1], [2]. Prior course designs add three architectural problems: *lock-in* to one proprietary LLM, *cost and privacy* exposure from sending every CV to a token-priced API, and *no evidence* of how much accuracy an open model would actually lose. Their KGs are also schema-light, with unverified LLM explanations.

We frame the task as *evidence-grounded, two-sided matching under a model-agnostic harness*: extract typed entities from CVs and postings into a shared ontology with source spans; normalize them to Lightcast skill and O*NET occupation IDs; rank jobs for a CV (and CVs for a job) better than lexical baselines; explain each match through satisfied requirements, gaps, and transferable-skill paths that cite graph edges; and act on gaps via CV edits, upskilling, company research, and outreach the user approves. Research questions: **RQ1** How much quality do open-weight models lose against a proprietary model, and what do they save in cost and latency? **RQ2** Can a LoRA-tuned small model match large zero-shot models on extraction? **RQ3** Does ontology-constrained extraction beat free-form subject–predicate–object (SPO) extraction on graph consistency and ranking? **RQ4** Do graph features improve ranking, and can explanations be fully grounded?

II. Data Sets

Jobs: Kaggle LinkedIn Job Postings 2023-24 (∼124K postings with job-skill, industry, company, and salary tables) [18], plus the Kaggle LinkedIn Job Posts Insights set (30K+) and Hugging Face `datastax` LinkedIn listings for cross-source robustness; Adzuna and RemoteOK feeds for freshness checks. **Resumes:** Kaggle Resume Dataset (∼2.4K PDFs, 24 categories) [18], the MIT-licensed GitHub `resume_corpus` (occupation labels), and our five CVs with consent. **Taxonomies:** Lightcast Open Skills, OSMT, and O*NET [19] for canonical IDs.

Derived data: Parsed-CV and Parsed-Job JSONL validated against a JSON Schema generated from the ontology (each field carries a source span, confidence, and extractor version); a raw SPO-triple file for the RQ3 ablation; a skill-alias table; a versioned Neo4j snapshot; and BM25/FAISS indexes. Test sets: 150 double-annotated documents for extraction, 40 CV profiles with pooled top-20 jobs judged 0/1/2 (800 pairs), and 60 explanations for faithfulness.

Efficient use: postings are deduplicated (title, company, MinHash of text) before any LLM call and outputs are cached by content hash. All postings get cheap processing (cleaning, BM25, embeddings); only a 20K stratified working set gets full LLM extraction. A large open model labels ∼4K documents, which we distill into a small LoRA model for the rest. Job-skill tables, resume categories, and O*NET occupation-skill lists supply free weak labels. Splits are fixed in Week 1, contact details are masked, CVs are processed on local Ollama, and hosted APIs see only anonymized Kaggle text.

III. State-of-Art Methods and Algorithms

Ranking. BM25 [3] and TF-IDF cosine are strong interpretable baselines, and Jaccard overlap of normalized skills is the simplest explainable fit score; all miss paraphrased skills. Sentence-BERT bi-encoders [4] embed CVs and postings in one space; cross-encoders rerank short lists [5]; Reciprocal Rank Fusion (RRF) merges ranked lists without calibration [6]; LambdaMART [7] learns to combine signals. **KG extraction.** LLMs can turn chunked text into SPO triples, then standardize entities and infer relations, as in the AI-Knowledge-Graph pipeline [8], but free-form predicates multiply and entities fragment. Ontology-guided extraction fills a fixed

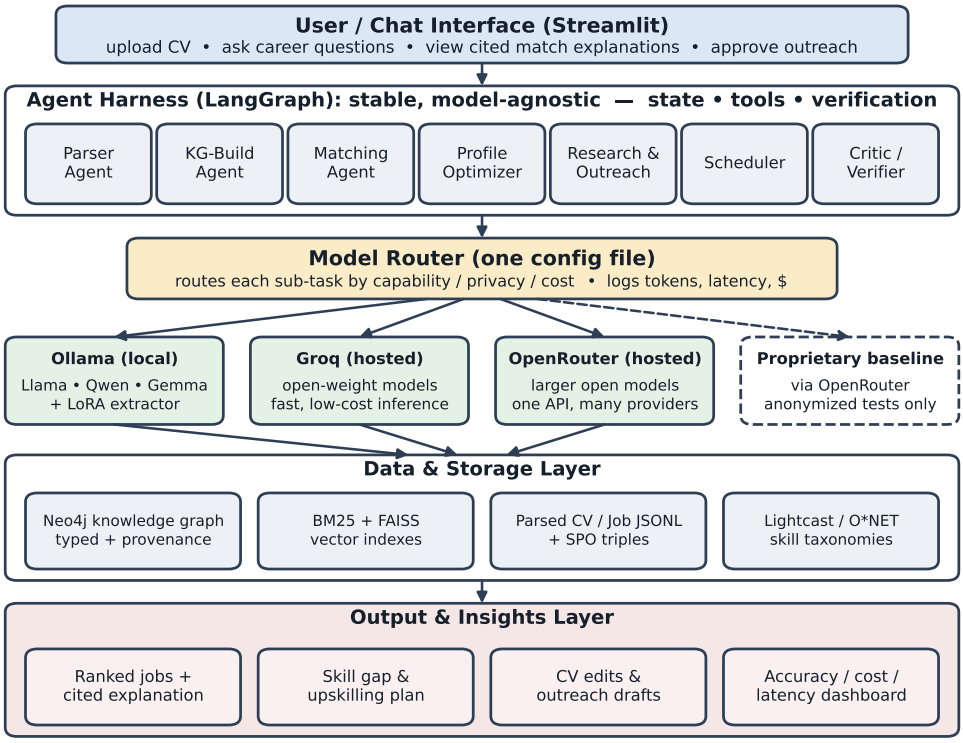


Fig. 1. System architecture: a stable LangGraph harness, a model router that logs tokens, latency, and cost per call, local and hosted open-weight tiers, and a dashed proprietary baseline used only for comparison.

schema instead; the course's starter-ontology method seeks the smallest ontology that still supports reasoning. GraphRAG grounds answers in graph neighbourhoods [9]. **Harnesses and open models.** A model supplies reasoning; the harness supplies orchestration, tools, state, and verification. Open coding harnesses such as OpenCode and Cline route among open models without rebuilding the workflow; we apply this to LangGraph [10]. Ollama [11] serves models locally behind an OpenAI-compatible API; Groq [12] offers fast hosted inference of open-weight models; OpenRouter [13] exposes many open and proprietary models through one OpenAI-compatible API, so the router's code is identical across tiers. **Adaptation.** LoRA [14] trains low-rank adapters on a frozen model; QLoRA [15] makes 7-8B models trainable on a single free Colab or Kaggle GPU.

IV. Research Plan

Seven threads; every modelling thread (R2–R6) names its *Classical baseline:* and its *Modern AI/DL:* so the comparison is explicit (summary in Table I; system in Fig. 1).

R1 Data and ontology. Ingest, deduplicate, and split the data; freeze an eleven-type ontology (Fig. 2) as JSON Schema: Candidate, JobPosting, Company, Occupation, Skill, Requirement (required/preferred, min. years), Experience, Credential, Project, Location, and Evidence (source span). Core relations: HAS_SKILL, HELD, USED, AT, POSTED_BY, FOR_ROLE, HAS_REQUIREMENT, TARGETS, RELATED_TO, TYPICAL_FOR, and SUPPORTED_BY, which every extracted edge must have. The structure mirrors the course's

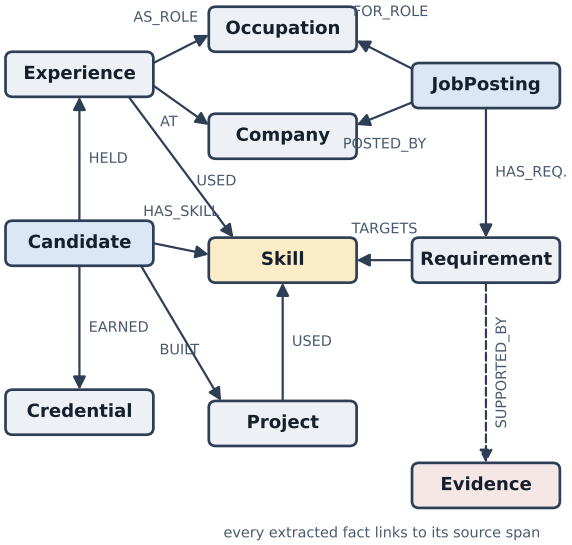


Fig. 2. Minimal job-market ontology (Location omitted). Every extracted edge is SUPPORTED_BY an Evidence span.

Land/Property Dispute reference: parties become Candidate and Company, claims become Requirements, evidence becomes source spans, and remedies become agent actions. R1 is shared infrastructure (MinHash dedupe, stratified sampling) and has no model comparison.

R2 Parsing benchmark (RQ1). *Classical baseline:* regex section segmentation plus a spaCy CRF NER model. *Modern AI/DL:* ontology-constrained JSON extraction by

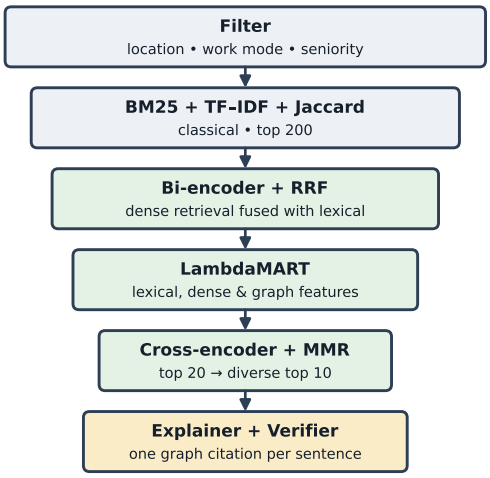


Fig. 3. Matching cascade: classical stages (grey) feed modern stages (green); the explainer (gold) may only state what the graph can cite.

open LLMs over 200-500-word chunks with 10% overlap, schema validation, and one repair retry; 3–4 open models across Ollama, Groq, and OpenRouter are compared with the proprietary baseline.

R3 Text to graph (RQ3). *Classical baseline:* rule-based and char-n-gram TF-IDF alias matching, Louvain communities [16]. *Modern AI/DL:* embedding nearest-neighbour normalization with LLM tie-breaking and LLM-proposed edges (`inferred=true`); Neo4j load via idempotent `MERGE`. The free-form SPO pipeline runs on the same documents as the ablation (Fig. 4).

R4 Matching cascade (RQ4). *Classical baseline:* filters (location, mode, seniority), then BM25, TF-IDF, and Jaccard top 200, plus logistic regression predicting occupation family; K-means for diversity. *Modern AI/DL:* bi-encoder retrieval fused by RRF, a LambdaMART ranker over lexical, dense, skill-coverage, experience-gap, and graph features (shortest skill path, RELATED_TO bridges to unmet requirements), and a cross-encoder on the top 20 with MMR. The cascade also runs job-to-candidate (Fig. 3).

R5 LoRA (RQ2). *Classical baseline:* the R2 CRF model. *Modern AI/DL:* QLoRA-tune a 7-8B model on distilled labels (100 human-corrected, held apart from the test set), serve it through Ollama, and compare it with zero-shot models on held-out F1 and latency.

R6 Harness and explanations (RQ4). *Classical baseline:* a template explainer that lists matched and missing skills. *Modern AI/DL:* seven LangGraph agents (Parser, KG-Build, Matching, Profile Optimizer, Research & Outreach, Scheduler, Critic) call the router, so a model swap is a one-line config change. Cypher returns satisfied requirements with evidence, gaps, and bridging paths; the LLM verbalizes only that evidence with an edge ID per sentence, and the Critic rejects uncited sentences. Gaps drive user-approved CV rewrites (rephrasing existing claims only), upskilling pointers, company briefs, outreach drafts, and Calendar follow-ups in a Streamlit UI.

TABLE I
CLASSICAL BASELINES VS. MODERN AI/DL, BY THREAD

Thread	Classical baseline	Modern AI/DL
R2 Extraction	Regex + spaCy CRF NER	Ontology-constrained open LLMs
R3 Normalization	Rules + char-n-gram TF-IDF	Embedding kNN + LLM tie-break
R4 Ranking	BM25, TF-IDF, Jaccard, logistic regression	Bi-encoder + RRF, LambdaMART with graph features, cross-encoder
R3/R4 Grouping	Louvain, K-means	MMR over embeddings
R5 Adaptation	CRF from R2	QLoRA-tuned 7-8B model
R6 Explanation	Template	Citation-constrained LLM + verifier

TABLE II
EVALUATION: OBJECTIVE, METRICS, AND TARGETS

Thread	Test set and metrics	Target
R1	Dedupe precision on 200 sampled pairs; split leakage check	$\geq$ 0.95; no overlap
R2, RQ1	150 docs (κ reported); per-type P/R/F1	F1 $\geq$ 0.80; $\geq$+10 over CRF
R3	500 mentions, top-1 accuracy; 100 sampled edges, precision	$\geq$ 0.90 each
R3, RQ3	Ontology vs. SPO: duplicate-node rate, ΔnDCG@10	Report both
R4, RQ4	40 profiles, pooled judgments; nDCG@10 [17], MRR, P@5; bootstrap test	+15% nDCG over BM25
R4 ablation	$-$graph features, $-$RRF, $-$cross-encoder	Report each
R5, RQ2	Same 150 docs; F1, latency vs. zero-shot teacher	Within 3 pts
R6, RQ4	60 explanations; faithfulness, hallucination rate	$\geq$ 0.95
R6	Blind A/B, 15–20 users, 1–5 Likert	Mean $\geq$ 4.0
R7, RQ1	Router logs; $/1K docs, p50/p95 latency	$<$3 s ranking

R7 Cost and latency (RQ1). A measurement thread: the router logs tokens, wall-clock time, and provider-reported cost for every call from Week 2, so RQ1 is answered from the runs that build the system.

V. EVALUATION PLAN

Each metric is tied to the thread and research question it tests (Table II). A modern component stays in the system only if it beats its classical baseline on fixed test splits. The headline RQ1 result is a model-by-task matrix of quality against cost per 1K documents and median latency for Ollama, Groq, OpenRouter, the LoRA model, and the proprietary baseline. Silver labels are used for training only.

VI. PROJECT TIMELINE

Milestones M1–M4 gate the build; a classical baseline runs end to end by Oct. 28 (Table III).

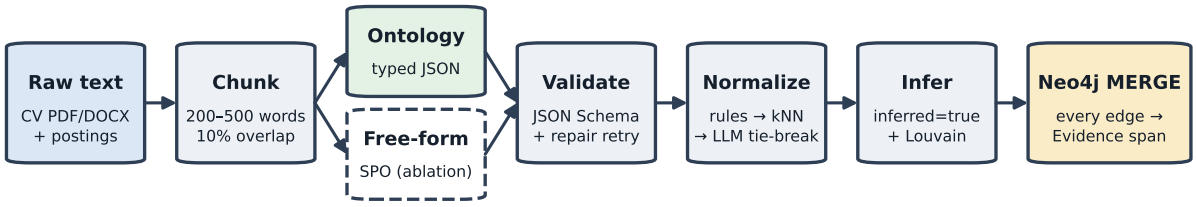


Fig. 4. Extraction-to-graph pipeline: ontology-constrained extraction is the main path; free-form SPO extraction (dashed) is the RQ3 ablation.

TABLE III
TIMELINE: TASKS, DESCRIPTIONS, DEADLINES (2026)

Dates	Tasks and description	Deadline
Oct 8-14	R1: data, splits, ontology schema, annotation guide	Oct 14 **M1**
Oct 15-21	R2: CRF baseline, LLM extraction, model comparison; router logging	Oct 21
Oct 22-28	R3 graph load; R4 BM25/TF–IDF/Jaccard end to end	Oct 28 **M2**
Oct 29-Nov 4	R5 QLoRA; dense + RRF; SPO ablation; relevance judging	Nov 4
Nov 5-11	LambdaMART, cross-encoder, explainer + verifier	Nov 11 **M3**
Nov 12-18	R6 agents, router config, Streamlit UI	Nov 18
Nov 19-25	R7 full evaluation, ablations, cost matrix	Nov 25 **M4**
Nov 26-Dec 2	User study, report, slides, demo video	Dec 2 final

VII. DIVISION OF WORK

Each member owns one module end to end and everyone annotates evaluation data. **Aryan Makhija:** ontology, Neo4j, explainer and verifier. **Krish Dhaval Naik:** data ingestion, job parser, live feeds, skill normalization. **Ashmit Amit Pai:** CRF baseline, LLM extraction, model benchmark, QLoRA, SPO ablation. **Himanshi Sunilbhai Ruparelia:** lexical baselines, dense + RRF, LambdaMART, cross-encoder, ranking evaluation. **Aditya Umesh Upadhyay:** LangGraph harness, model router (Ollama, Groq, OpenRouter), agents, UI, cost/latency logging, user study.

REFERENCES

[1] D. Ç. Ertuğrul and S. Bitirim, "Job recommender systems: A systematic literature review, applications, open issues, and challenges," *J. Big Data*, vol. 12, no. 140, 2025.

[2] C. de Ruijt and S. Bhulai, "Job recommender systems: A review," arXiv:2111.13576, 2021.

[3] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Found. Trends Inf. Retr.*, vol. 3, no. 4, 2009.

[4] N. Reimers and I. Gurevych, "Sentence-BERT," in *Proc. EMNLP-IJCNLP*, 2019.

[5] R. Nogueira and K. Cho, "Passage re-ranking with BERT," arXiv:1901.04085, 2019.

[6] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. SIGIR*, 2009.

[7] C. J. C. Burges, "From RankNet to LambdaRank to LambdaMART: An overview," MSR-TR-2010-82, 2010.

[8] R. McDermott, "AI-Knowledge-Graph," GitHub, https://github.com/robert-mcdermott/ai-knowledge-graph

[9] D. Edge *et al.*, "From local to global: A Graph RAG approach to query-focused summarization," arXiv:2404.16130, 2024.

[10] LangChain, "LangGraph," https://langchain-ai.github.io/langgraph/

[11] Ollama, https://github.com/ollama/ollama

[12] Groq, "GroqCloud documentation," https://console.groq.com/docs

[13] OpenRouter, "Documentation," https://openrouter.ai/docs

[14] E. J. Hu *et al.*, "LoRA: Low-rank adaptation of large language models," in *Proc. ICLR*, 2022.

[15] T. Dettmers *et al.*, "QLoRA: Efficient finetuning of quantized LLMs," in *Proc. NeurIPS*, 2023.

[16] V. D. Blondel *et al.*, "Fast unfolding of communities in large networks," *J. Stat. Mech.*, P10008, 2008.

[17] K. Järvelin and J. Kekäläinen, "Cumulated gain-based evaluation of IR techniques," *ACM TOIS*, vol. 20, no. 4, 2002.

[18] Kaggle, "LinkedIn Job Postings (2023–2024)" and "Resume Dataset," https://www.kaggle.com/datasets/arshkon/linkedin-job-postings, https://www.kaggle.com/datasets/snehaanbhawal/resume-dataset

[19] Lightcast Open Skills, https://lightcast.io/open-skills; O*NET, https://www.onetcenter.org